Práctica S08: Resiliencia y Transacciones Distribuidas en Arquitectura de Microservicios (Mexi-Banco)

Alumno: Diego Alexander Martínez Pérez

1. Introducción y Arquitectura Base

El presente reporte documenta la implementación de patrones de resiliencia y transacciones distribuidas para el sistema bancario "Mexi-Banco". La arquitectura está basada en microservicios desarrollados con Spring Boot, orquestados mediante un servidor de descubrimiento (Netflix Eureka) y centralizados a través de un Spring Cloud Gateway que actúa como único punto de entrada (puerto 8080). Cada microservicio cuenta con su propia base de datos PostgreSQL, garantizando el aislamiento de los datos. Para la comunicación inter-servicios se utiliza un RestTemplate con balanceo de carga (@LoadBalanced), permitiendo la resolución de nombres lógicos en lugar de direcciones IP estáticas.

2. Implementación de Transacciones Distribuidas (Patrón Saga)

En un ecosistema de microservicios, las transacciones que abarcan múltiples dominios (como transferir saldo de la cuenta A a la cuenta B) no pueden depender de bloqueos ACID tradicionales. Para resolver esto, se implementó el Patrón Saga mediante coreografía/orquestación.

Escenario de Prueba:

Se configuró una regla de negocio donde ciertas cuentas se catalogan como "tóxicas" (ej. CLABE S0721791802560). Al intentar transferir fondos hacia esta cuenta, el microservicio cuenta rechaza el abono simulando una falla o restricción.

Mecanismo de Compensación:

Al detectar el fallo en el abono, el microservicio de transferencia captura la excepción e inicia una transacción de compensación. Se emite una orden de reverso (cargo) a la cuenta origen para devolver los fondos retenidos, asegurando la consistencia eventual del sistema sin dejar el dinero "perdido" en el limbo.

```
PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> curl.exe -i -X POST "http://localhost:8080/api/transferencias" -H
"Content-Type: application/json" -d "{\`"claveOrigen\`":\`"$CLABE_A\`",\`"claveDestino\`":\`"$CLABE_B\`",\`"monto\`":200
}"
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 16:15:09 GMT

{"timestamp":"2026-10-19T16:15:09.430+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/$CLABE_A
"

id clabe          titular       saldo
-- -----          -------       -----
 2 S0711791802554 Diego Origen 700.00


PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/$CLABE_B
"

id clabe          titular        saldo
-- -----          -------        -----
 3 S0721791802560 Diego Destino 400.00


PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> curl.exe -s http://localhost:8080/api/transferencias/1
PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> curl.exe -s http://localhost:8080/api/transferencias/2
```

3. Tolerancia a Fallos (Circuit Breaker Personalizado)

Para prevenir fallos en cascada y saturación de red cuando el servicio de cuentas se degrada, se inyectó una implementación manual de Circuit Breaker (clase CircuitBreakerCuenta) dentro del componente CuentaCliente en el microservicio de transferencias. Este mecanismo gestiona tres estados mediante una máquina de estados finitos:

3.1. Estado CLOSED (Operación Normal)

El sistema opera sin interrupciones. Las peticiones de saldo y transferencias fluyen desde el transferencia hacia las réplicas de cuenta. La tasa de fallos se mantiene por debajo del umbral establecido.

```
PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> curl.exe -i -X POST "http://localhost:8080/api/transferencias" -H "Content-Type: application/json" -d
"{\`"claveOrigen\`":\`"$CLABE_A\`",\`"claveDestino\`":\`"$CLABE_B\`",\`"monto\`":200}"
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 16:15:09 GMT

{"timestamp":"2026-10-19T16:15:09.430+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/$CLABE_A"

id clabe          titular       saldo
-- -----          -------       -----
 2 S0711791802554 Diego Origen 700.00

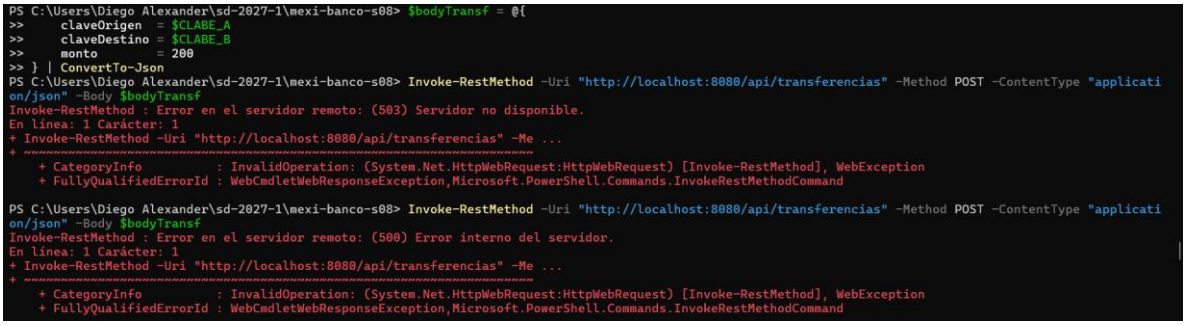

PS C:\Users\Diego Alexander\sd-2027-1\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/$CLABE_B"

id clabe          titular        saldo
-- -----          -------        -----
 3 S0721791802560 Diego Destino 400.00
```

3.2. Estado OPEN y Patrón Fast-Fail

Al detener intencionalmente el contenedor cuenta (docker compose stop cuenta), las llamadas de red comienzan a fallar. Al alcanzar el umbral máximo de fallos consecutivos configurado, el circuito transiciona al estado OPEN.

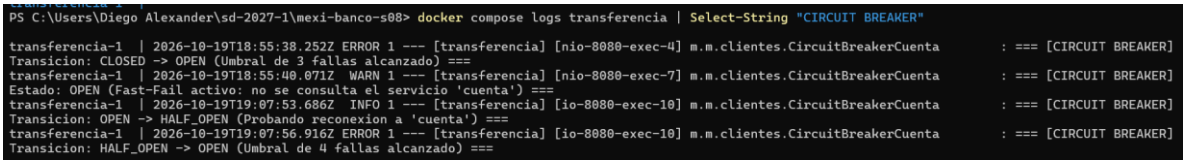
A partir de este momento, se activa el comportamiento Fast-Fail: el microservicio de transferencias rechaza inmediatamente cualquier nueva petición con un error 503 Service Unavailable, protegiendo los recursos del hilo de ejecución y evitando tiempos de espera inútiles (timeouts) hacia un servicio que ya se sabe que está caído.



3.3. Estado HALF_OPEN (Autorrecuperación)

El Circuit Breaker cuenta con un temporizador de enfriamiento (10 segundos). Al expirar este tiempo, el circuito transiciona automáticamente al estado HALF_OPEN. En este estado, se permite el paso de una cantidad limitada de peticiones de prueba para verificar si el microservicio cuenta ya se ha recuperado.

- Si la petición de prueba falla, el circuito se reabre de inmediato (OPEN), protegiendo nuevamente el sistema.
- Si la petición de prueba es exitosa, el contador de fallos se reinicia y el circuito se cierra por completo (CLOSED), restaurando la normalidad.



4. Conclusiones

La práctica demuestra que en una arquitectura de microservicios, el diseño orientado a fallos no es opcional. La integración del patrón Saga garantiza la integridad financiera y de los datos a pesar de las interrupciones en los flujos distribuidos. Por su parte, la construcción manual

del Circuit Breaker evidenció cómo el aislamiento de fallos (Fast-Fail) es crítico para mantener la resiliencia global de la plataforma, evitando que la caída de un servicio secundario derribe el nodo principal por agotamiento de recursos.